

PROJECT REPORT

Late Arriving Transaction Data Processing Pipeline Using Databricks

1. Project Title

Real-Time Transaction Data Processing Using Databricks Auto Loader, Delta Lake, Medallion Architecture, and Streaming Watermark

2. Project Overview

This project implements an end-to-end data engineering pipeline using Databricks. The pipeline processes transaction data through different layers of the Medallion Architecture:

- Bronze Layer for raw data ingestion
- Silver Layer for data cleaning and transformation
- Gold Layer for business-level reporting

The project also handles late-arriving transactions using transaction date and ingestion date comparison. Delta Lake MERGE operations are used to update impacted records. Structured Streaming Watermark is implemented to manage late data efficiently.

3. Objective

The main objectives of this project are:

- Ingest transaction files automatically using Databricks Auto Loader
- Store raw data in Bronze Delta tables
- Clean and validate data in Silver Layer
- Generate daily revenue reports in Gold Layer
- Detect late-arriving transactions
- Update historical data using Delta MERGE
- Implement streaming watermark for late data handling
- Perform data quality validation checks

4. Technologies Used

Technology	Purpose
Databricks	Data processing platform
Apache Spark	Distributed data processing
PySpark	Data transformation
Delta Lake	Storage and ACID transactions
Auto Loader	Incremental file ingestion
Structured Streaming	Real-time processing
Watermark	Late event handling

5. Project Architecture

1. CSV Transaction File

2. Databricks Auto Loader

3. Bronze Delta Table
(bronze_transactions)

4. Silver Transformation
(Data Cleaning & Validation)

5. Silver Delta Table
(silver_transactions)

6. Gold Aggregation Daily Revenue

7. Late Transaction Detection

8.gold_daily_revenue

9. Delta MERGE Update

10. Watermark Streaming

11. watermark_gold_revenue

6. Bronze Layer Implementation

Purpose

The Bronze layer stores raw transaction data ingested from CSV files.

Implementation

Databricks Auto Loader was used for incremental file ingestion.

Process:

1. CSV File
2. Auto Loader
3. Bronze Delta Table

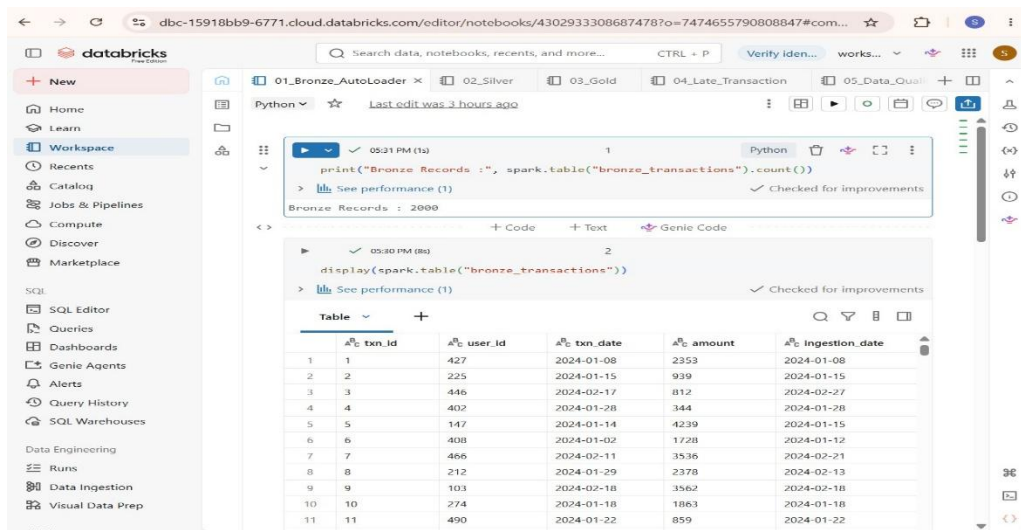
Table Created:

bronze_transactions

Features implemented:

- Automatic file detection
- Schema inference
- Schema storage using schema location
- Checkpoint management

Bronze Auto Loader



The screenshot shows the Databricks workspace interface. The notebook is titled '01_Bronze_AutoLoader'. It contains two code cells. The first cell, executed at 05:21 PM, prints 'Bronze Records : 2000'. The second cell, executed at 05:30 PM, displays a table of transaction data. The table has columns: txn_id, user_id, txn_date, amount, and ingestion_date. The data is as follows:

txn_id	user_id	txn_date	amount	ingestion_date
1	427	2024-01-08	2353	2024-01-08
2	225	2024-01-15	939	2024-01-15
3	446	2024-02-17	812	2024-02-27
4	402	2024-01-28	344	2024-01-28
5	147	2024-01-14	4239	2024-01-15
6	408	2024-01-02	1728	2024-01-12
7	466	2024-02-11	3536	2024-02-21
8	212	2024-01-29	2378	2024-02-13
9	103	2024-02-18	3562	2024-02-18
10	274	2024-01-18	1863	2024-01-18
11	490	2024-01-22	859	2024-01-22

7. Silver Layer Implementation

Purpose

The Silver layer performs data cleaning and transformation.

Operations performed:

- Data type conversion
- Duplicate removal
- Invalid record filtering
- Data validation

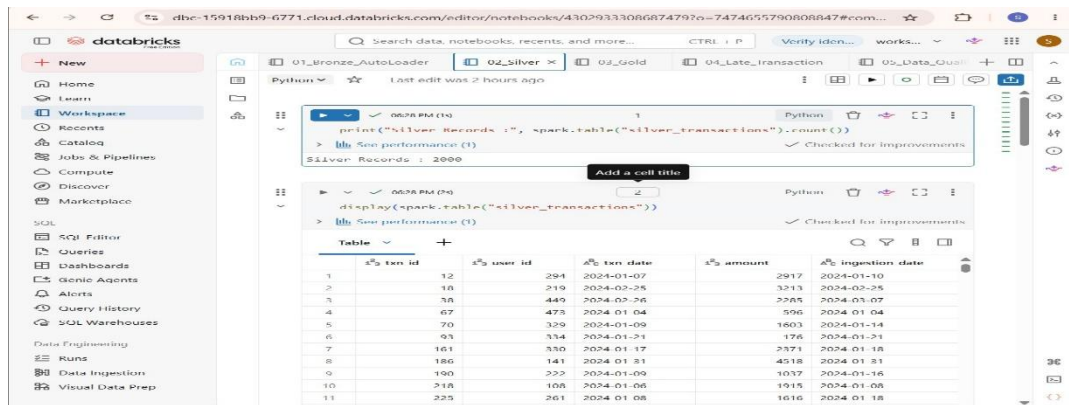
Example transformations:

- txn_id converted from string to integer
- user_id converted from string to integer
- amount converted from string to integer

Table Created:

silver_transactions

Silver notebook



The screenshot shows a Databricks notebook interface. The left sidebar contains navigation options like Home, Learn, Workspace, Recents, Catalog, Jobs & Pipelines, Compute, Discover, Marketplace, SQL Editor, Queries, Dashboards, Genie Agents, Alerts, Query History, and SQL Warehouses. The main area displays a Python code cell with the following code:

```
print("Silver Records :", spark.table("silver_transactions").count())
```

The output shows "Silver Records : 2000". Below the code, a table is displayed with the following columns: `txn_id`, `user_id`, `txn_date`, `amount`, and `ingestion_date`. The table contains 11 rows of data.

	<code>txn_id</code>	<code>user_id</code>	<code>txn_date</code>	<code>amount</code>	<code>ingestion_date</code>
1	12	294	2024-01-07	2917	2024-01-10
2	16	219	2024-02-26	3213	2024-02-26
3	38	449	2024-02-26	2268	2024-03-07
4	67	473	2024-01-04	596	2024-01-04
5	70	329	2024-01-09	1603	2024-01-14
6	93	334	2024-01-21	176	2024-01-21
7	161	330	2024-01-17	2371	2024-01-18
8	186	141	2024-01-31	4518	2024-01-31
9	190	222	2024-01-09	1037	2024-01-16
10	218	108	2024-01-06	1915	2024-01-08
11	225	261	2024-01-08	1616	2024-01-18

8. Gold Layer Implementation

Purpose

The Gold layer creates business-ready analytical data.

Daily revenue calculation was performed by grouping transactions based on transaction date.

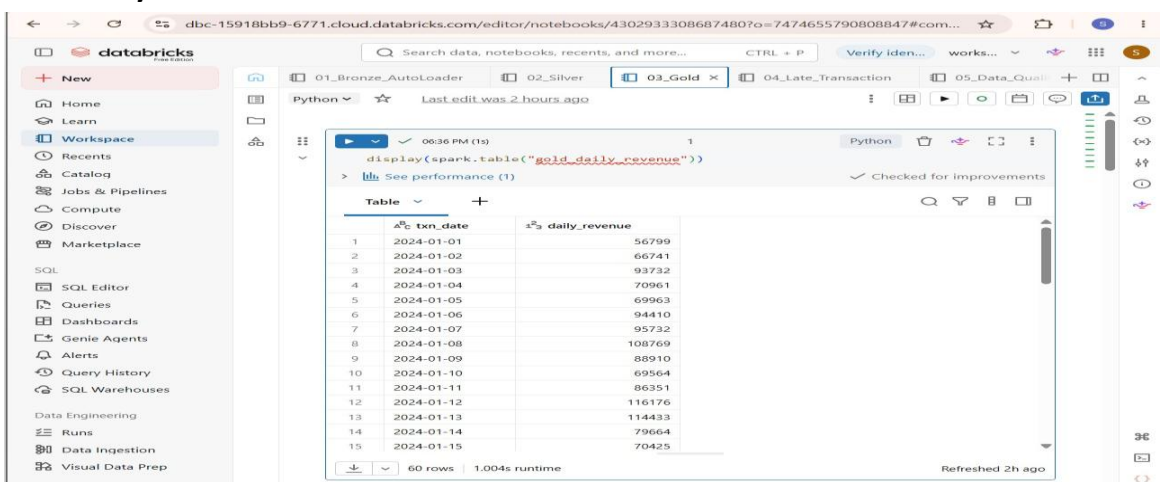
Logic:

1. `txn_date`
2. `SUM(amount)`
3. `daily_revenue`

Table Created:

`gold_daily_revenue`

Gold Layer



The screenshot shows a Databricks notebook interface. The left sidebar contains navigation options like Home, Learn, Workspace, Recents, Catalog, Jobs & Pipelines, Compute, Discover, Marketplace, SQL Editor, Queries, Dashboards, Genie Agents, Alerts, Query History, and SQL Warehouses. The main area displays a Python code cell with the following code:

```
display(spark.table("gold_daily_revenue"))
```

The output shows a table with the following columns: `txn_date` and `daily_revenue`. The table contains 15 rows of data.

	<code>txn_date</code>	<code>daily_revenue</code>
1	2024-01-01	56799
2	2024-01-02	66741
3	2024-01-03	93732
4	2024-01-04	70961
5	2024-01-05	69963
6	2024-01-06	94410
7	2024-01-07	95732
8	2024-01-08	108769
9	2024-01-09	88910
10	2024-01-10	69564
11	2024-01-11	86351
12	2024-01-12	116176
13	2024-01-13	114433
14	2024-01-14	79664
15	2024-01-15	70425

9. Late Arriving Transaction Handling

Objective

Identify transactions where data arrives after the actual transaction date.

Condition used:

txn_date < ingestion_date

Late transactions detected:

1415 records

Process:

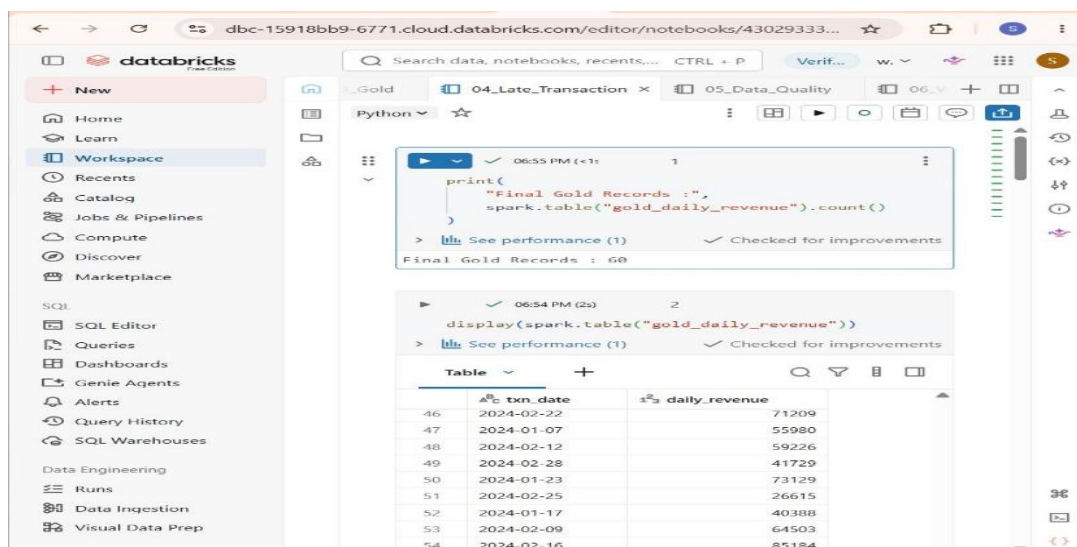
1. Read Silver table
2. Compare transaction date and ingestion date
3. Identify late transactions
4. Recalculate affected revenue
5. Update Gold table using Delta MERGE

Delta MERGE

MERGE operation was used to:

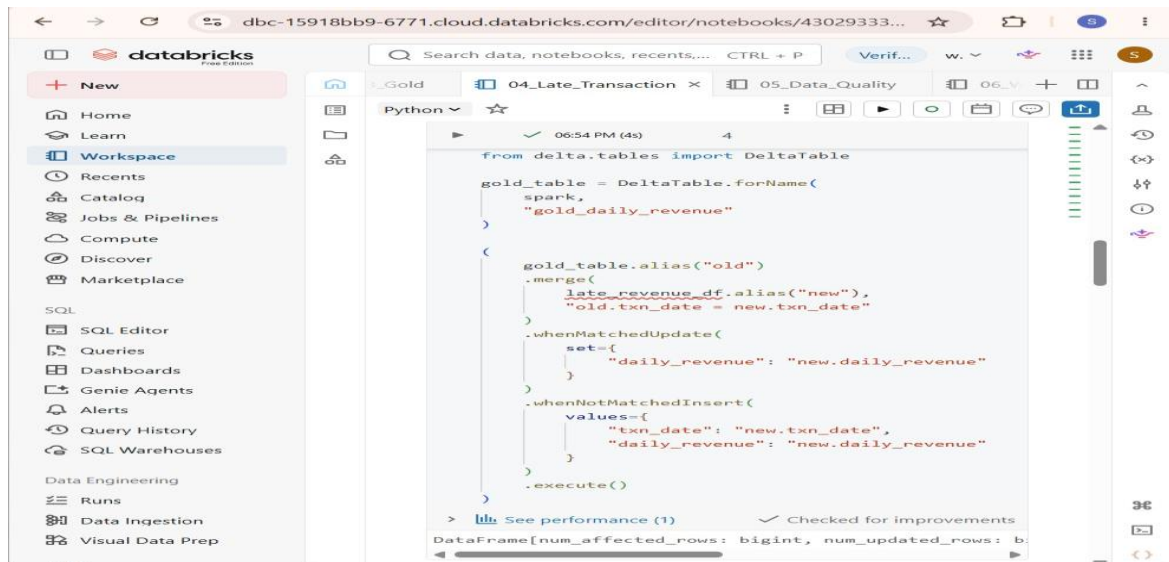
- Update existing transaction dates
- Insert new transaction dates

Late Arriving Transaction Handling & Delta MERGE



The screenshot shows the Databricks workspace interface. The left sidebar contains navigation options like Home, Learn, Workspace, Catalog, Jobs & Pipelines, Compute, Discover, and Marketplace. The main area displays a Python notebook with two code cells. The first cell prints the final gold records count, which is 60. The second cell displays the gold_daily_revenue table. Below the code cells, a table view shows the data for the gold_daily_revenue table, including columns for txn_date and daily_revenue.

Table	txn_date	daily_revenue
46	2024-02-22	71209
47	2024-01-07	55980
48	2024-02-12	59226
49	2024-02-28	41729
50	2024-01-23	73129
51	2024-02-25	26615
52	2024-01-17	40388
53	2024-02-09	64503
54	2024-02-16	85184



10. Watermark Streaming Implementation

Purpose

Watermark was implemented using Structured Streaming to handle late-arriving events.

Implementation steps:

- Read Bronze Delta table as streaming source
- Convert transaction date into timestamp format
- Apply watermark on event time
- Perform window aggregation
- Store output into Delta table

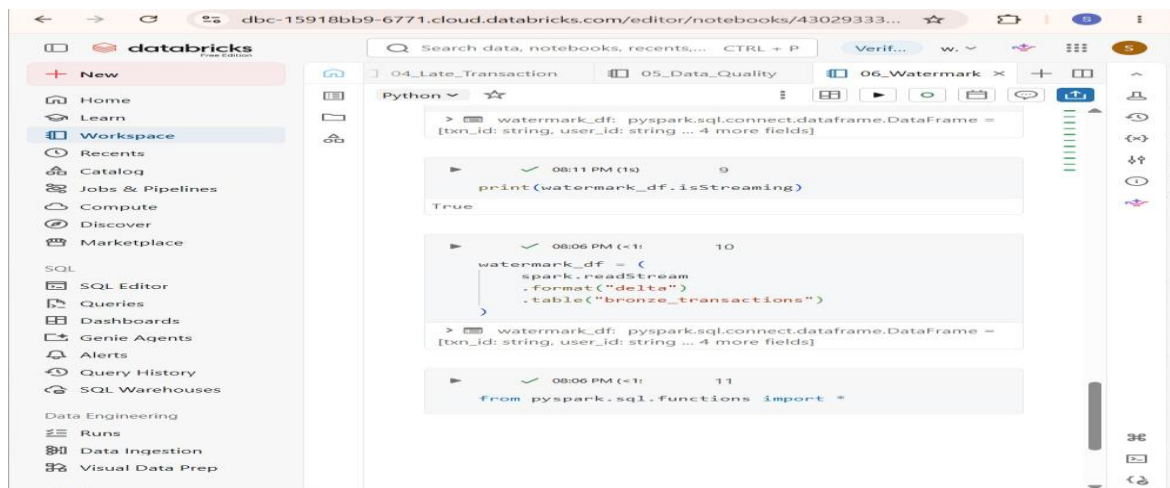
Watermark configuration:

1. txn_date
2. 1 day delay

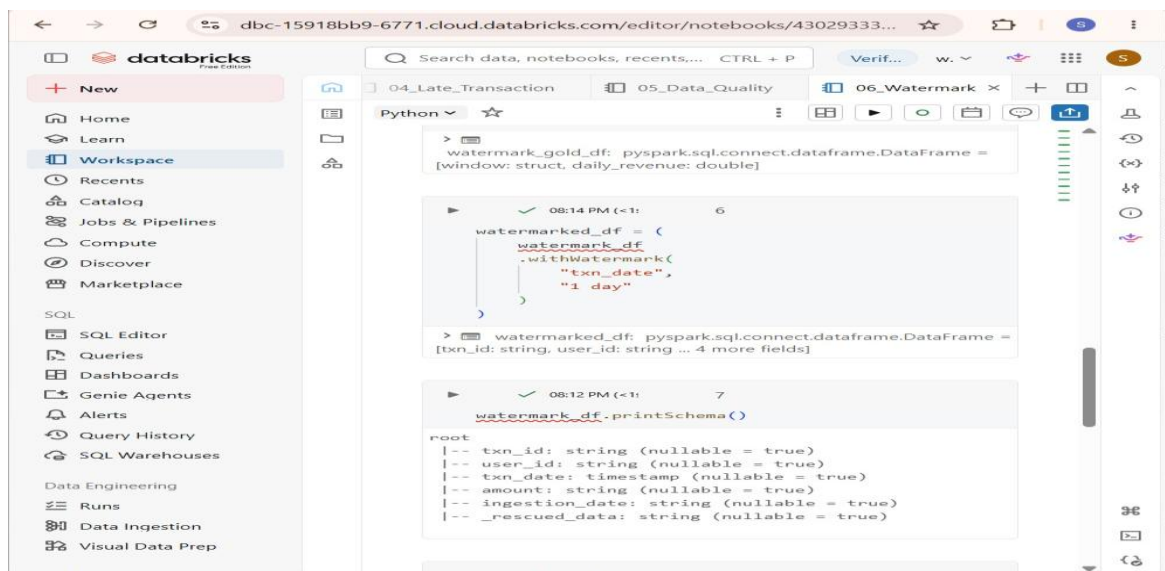
Table Created:

watermark_gold_revenue

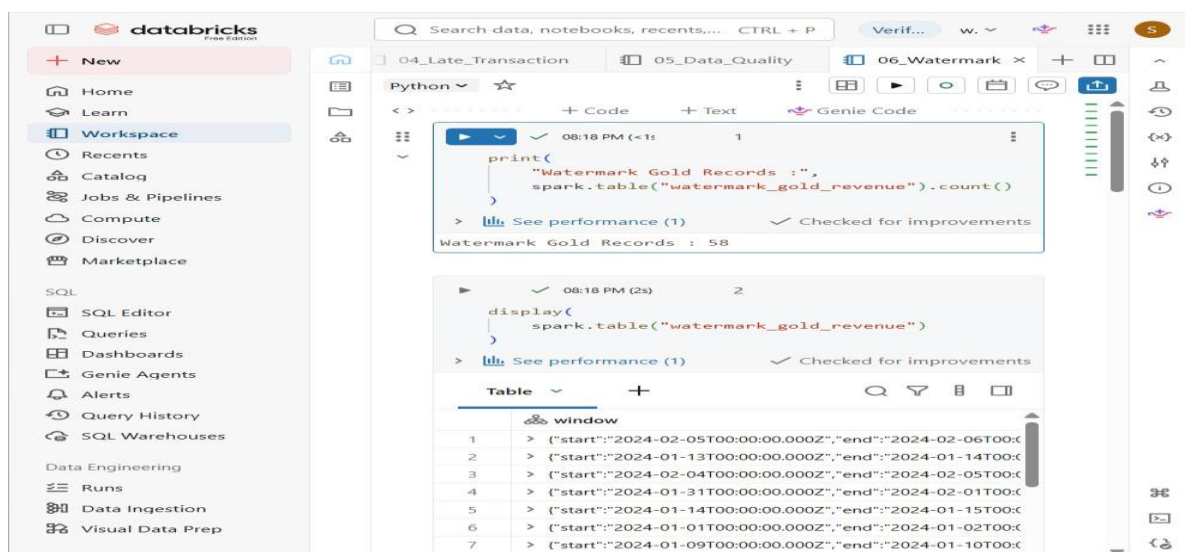
Watermark Streaming



With Watermark



Watermark Gold Records



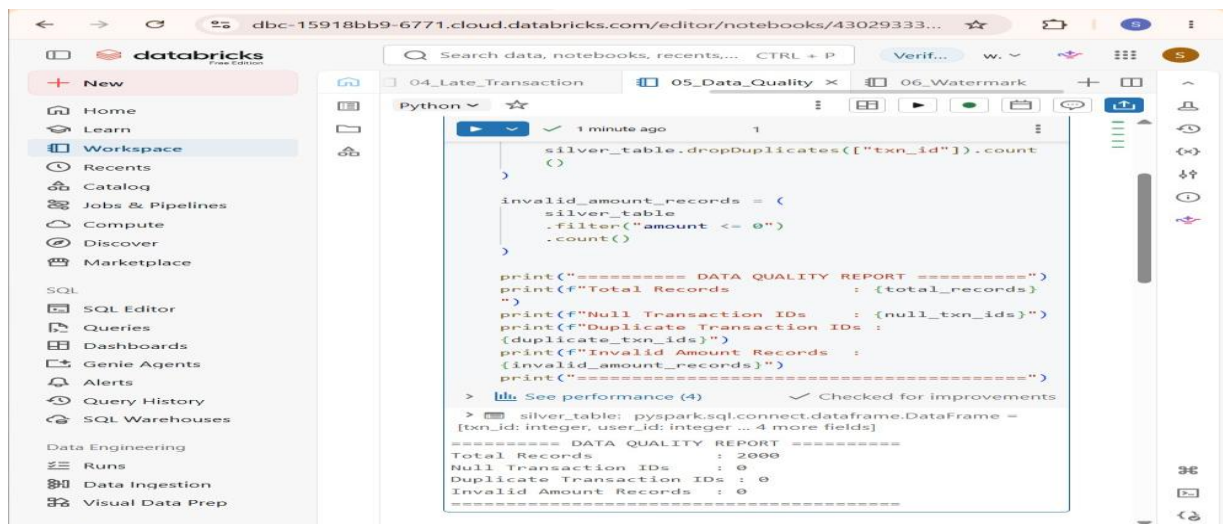
11. Data Quality Validation

Data quality checks were performed on the Silver layer.

Checks performed:

Validation	Result
Total Records	2000
Null Transaction IDs	0
Duplicate Transaction IDs	0
Invalid Amount Records	0

Data Quality



```
silver_table.dropDuplicates(["txn_id"]).count()

invalid_amount_records = (
    silver_table
    .filter("amount <= 0")
    .count()
)

print("===== DATA QUALITY REPORT =====")
print(f"Total Records : {total_records}")
print(f"Null Transaction IDs : {null_txn_ids}")
print(f"Duplicate Transaction IDs : {duplicate_txn_ids}")
print(f"Invalid Amount Records : {invalid_amount_records}")
print("=====")

> See performance (4) ✓ Checked for improvements
> silver_table: pyspark.sql.connect.dataframe.DataFrame = [txn_id: integer, user_id: integer ... 4 more fields]
===== DATA QUALITY REPORT =====
Total Records : 2000
Null Transaction IDs : 0
Duplicate Transaction IDs : 0
Invalid Amount Records : 0
=====
```

12. Final Tables Created

The following Delta tables were created:

Table Name	Layer
Bronze_transaction	Bronze
Silver_transaction	Silver
Gold_daily_revenue	Gold
Watermark_gold_revenue	Streaming Gold

13. Conclusion

This project successfully implements an end-to-end Databricks data engineering pipeline using Medallion Architecture.

The solution demonstrates:

- Automated ingestion using Auto Loader
- Data transformation using PySpark
- Delta Lake table management
- Business reporting using Gold layer
- Late-arriving data processing using Delta MERGE
- Streaming data handling using Watermark
- Data quality validation

The pipeline is scalable, reliable, and suitable for real-world transaction processing systems.

Appendix

Screenshots attached:

1. Workspace Structure
2. Volume Structure
3. Bronze Layer
4. Silver Layer
5. Gold Layer
6. Late Transaction Handling
7. Watermark Streaming
8. Data Quality Results
9. Delta Tables